

Operación Tarasca: Laberinto de Volantes

Autores: María Megías Moyano y Martín Hernández Ruiz

Asignatura: Sistemas Gráficos (SG)

Página 0 / 0

Operación Tarasca: Laberinto de Volantes

Autores: María Megías Moyano y Martín Hernández Ruiz

Resumen

Proyecto de un juego en primera persona donde el jugador explora un laberinto ambientado en una feria andaluza. Debe recoger una serie de pick-ups para conseguir la llave que abre la puerta final y completar la partida. La memoria describe las técnicas de modelado, animación, algoritmos de movimiento y colisión, y la arquitectura básica del juego.

1. Portada

Operación Tarasca: Laberinto de Volantes

Autores: María Megías Moyano y Martín Hernández Ruiz

Curso: Ingeniería Informática — Práctica SG

2. Documento de la Defensa 1 corregido

Aquí se adjunta la versión corregida de la plantilla inicial del juego. Se han incorporado las siguientes correcciones solicitadas en la defensa:

- Ajustes en la lógica de recogida de pick-ups para evitar recogidas involuntarias.
- Mejora del control de colisiones para impedir que el jugador atraviese las paredes al moverse en diagonal.
- Pulido visual del material del farolillo y corrección de la posición del pomo de la puerta.

Incorporación de documentos de defensa y notas "fuera de temario"

Se han integrado el contenido y correcciones recogidas en los ficheros de defensa y justificación del proyecto:

- Defensa3.md: detalles de la implementación del laberinto (Laberinto/Laberinto.js), la comprobación de colisiones (método puedeMoverseA() que valida cuatro puntos alrededor del jugador para simular volumen), la creación y posicionamiento de pick-ups y las animaciones del Abanico, Castañuelas y Rebujito (líquido e hielos).

Página 0 / 0

- Defensa4.md: descripción de la OrthographicCamera para el minimapa, la integración del minimapa con setViewport/setScissor, y el detalle de materiales (uso de CanvasTexture como bumpMap, MeshPhysicalMaterial para vidrio) y luces (ambiental, direccional, puntual y dinámica).
- FueraTemario.md: justificación de herramientas y técnicas fuera del temario (CSG con three-bvh-csg, dat.GUI, OrbitControls como visor auxiliar, CanvasTexture, FileLoader para laberinto.txt, y teclas de depuración F y P).

Estos documentos han servido para enriquecer las explicaciones técnicas del presente documento y aportar fragmentos de código y justificaciones que se han insertado en las secciones pertinentes de esta memoria.

3. Descripción general del juego

- Temática: el escenario es una feria tradicional con casetas y ornamentación andaluza. El laberinto recrea pasillos de feria y casetas con ambientación diurna/nocturna.
- Objetivo: encontrar y recoger todos los pick-ups repartidos por el laberinto; una vez reunidos, usar la llave para abrir la puerta final y completar la partida.
- Perspectiva: juego en primera persona con control mediante teclado y ratón.
- Mecánica central: recoger todos los pick-ups desbloquea la apertura de la puerta; la interacción con el pomo (mediante raycast y click) inicia la animación de apertura.

4. Pick-ups implementados

Se han implementado cuatro pick-ups principales: Farolillo (llave), Abanico articulado, Castañuelas y Rebujito (vaso). A continuación se describen técnicas y detalles de cada uno.

4.1 Farolillo / llave

- Función: actúa como la "llave" que permite la apertura de la puerta final cuando se coloca en la cerradura.
- Técnicas de modelado utilizadas:
 - Revolución: para las piezas simétricas circulares del farolillo (estructura cilíndrica básica).
 - Extrusión: para detalles planos como las aletas o remaches decorativos.
 - CSG (operaciones booleanas): para tallar el hueco interior y crear detalles del farolillo.

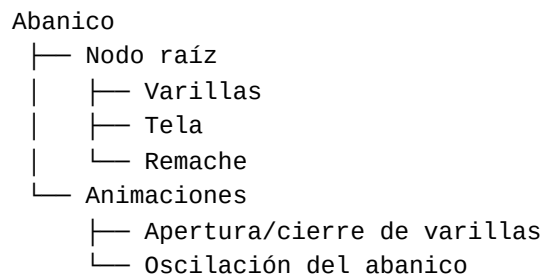
Página 0 / 0

- Textura de lunares y luz interior: se aplica una textura en el canal de color y se añade una fuente de luz interior para simular iluminación cálida.

4.2 Abanico articulado

- Función: pick-up animado, elemento articulado con partes móviles jerárquicas.
- Técnicas: modelado por barrido/extrusión para las varillas y la tela; uso de jerarquía de nodos para animar apertura/cierre.

Modelo jerárquico (obligatorio según el guion):



- La estructura jerárquica permite aplicar rotaciones locales a cada varilla, produciendo la apertura coherente del abanico. La tela está ligada a las varillas para seguir el movimiento.
- Animación: tweening continuo para oscilación y animación por pasos para apertura/cierre.

4.3 Castañuelas

- Técnicas: revolución para la forma principal, extrusión para las "orejas" o detalles, CSG para crear el hueco interior que simula la unión entre piezas, y creación de un cordel por extrusión simple.
- Material: acabado con aspecto de madera mediante mapas de color y ajuste de roughness/metalness para simular madera mate.

4.4 Vaso / Jarra de Rebujito

- Técnicas: revolución para la geometría del vaso/jarra; barrido (sweep) para la pajita; extrusión para la rodaja de limón.
- Materiales: vidrio transparente con índice de refracción simulado mediante valores bajos de metalness y alta transparencia; líquido con un material translúcido y pequeños cubos para simular hielo.

5. Laberinto y puerta final

- Construcción de muros: el laberinto se genera a partir de un mapa en `laberinto.txt` que se parsea y posiciona paredes y huecos en una rejilla. Cada celda se traduce a mundo 3D mediante la función `getMundoFromCelda`.
- Ubicación de pick-ups: los pick-ups se colocan en celdas fijadas en el mapa usando `posicionarPickup(obj, fila, columna)`, ajustando escalas y centros visuales para coherencia.
- Puerta con pomo: la puerta está compuesta por un `doorGroup` y un `doorPivot` que rota la hoja; el pomo (`doorKnob`) es un mesh interactuable marcado con `userData.interactable = 'door'`.
- Condición de apertura: la función `todosPickupsRecogidos()` evalúa si todos los pick-ups han sido recogidos; la apertura solo se inicia cuando esa condición es verdadera y el jugador está suficientemente cerca del pomo.

6. Algoritmos usados

A continuación se documentan los algoritmos implementados, con explicación técnica y referencias al código.

6.1 Movimiento por el laberinto

- Control: el jugador se mueve en primera persona usando `PointerLockControls` para la orientación con el ratón y teclas W/S (adelante/atrás) para el avance.
- Implementación: la dirección de avance se obtiene con `cameraControl.getDirection(this.tmpDirection)` y se proyecta al plano XZ (anulando Y). El desplazamiento se escala por velocidad * delta y se aplica por componentes (`tryMoveAxis`) para permitir deslizamiento sobre paredes.

6.2 Colisiones con muros

- Método: antes de mover la cámara se construye una candidate (posición destino). Se valida mediante `this.model.puedeMoverseA(position, this.playerRadius)` y comprobando además que no colisione con pick-ups (función `intersectsPickupObstacle`).
- Razonamiento: al validar la posición antes de aplicar el movimiento evitamos penetrar geometría y solucionamos problemas con movimiento diagonal.

6.3 Recogida de pick-ups

- Detección: se usa un `Raycaster` que se lanza desde el centro de la pantalla (`centerPointer`) o desde la posición del ratón cuando el cursor está libre.

- Condición: la función `tryPickUp` comprueba intersecciones contra `this.pickups`; si la distancia es menor que `interactionDistance` y el objeto tiene `userData.recogible` y no ha sido recogido, se invoca `recogerObjeto`.
- Efecto: al recoger, el pick-up se marca (`recogido = true`), se oculta (`visible = false`) y se incrementa el contador interno `pickupsRecogidos`.

6.4 Apertura de puerta

La apertura de la puerta requiere dos condiciones:

```
todosPickupsRecogidos == true  
distanciaJugadorPomo < interactionDistance
```

- Comprobación: `tryInteractWithDoor` lanza un raycast contra `doorKnob` y valida la distancia. Si faltan pick-ups se muestra mensaje y feedback lumínico; si están todos, se inicia la secuencia `animateKeyUnlock()` → `startDoorOpening()` que anima la llave y la rotación de la hoja.

7. Cámaras

- Cámara principal: `PerspectiveCamera` en primera persona, posicionada a `playerHeight` sobre el terreno.
- Cámara superior: `OrthographicCamera` usada para el mini-mapa; su vista se mantiene en una capa separada y su frustum se ajusta para cubrir todo el laberinto (`configureTopCamera`).
- Visualización: la cámara superior se muestra como un minimapa en pantalla cuando `mostrarMiniMapa` está activo (controlable desde la GUI).

8. Materiales, texturas y luces

- Materiales: se han utilizado `MeshStandardMaterial` para la mayoría de objetos, ajustando `roughness`, `metalness`, `emissive` y mapas de color cuando procede.
- Texturas: algunas piezas usan texturas en el canal de color (por ejemplo lunares del farolillo). Las normales se simulan con texturas generadas por canvas en algunas superficies (suelo y normal map para detalles).
- Iluminación: mezcla de `AmbientLight`, `DirectionalLight` (sol) y `PointLight` dinámicos. Se implementa una `dynamicLight` con cambio de color en el tiempo para dar vida a la escena.

9. Animaciones

- Abanico: animación continua de oscilación y apertura/cierre ejecutada mediante tweens y actualizaciones por frame.
- Puerta: animación de llave (inserción y giro) seguida de interpolación de rotación de `doorPivot` con `TWEEN`.
- Farolillo: emisión de luz interna y posible animación de intensidad.

10. Manual de Usuario

Requisitos mínimos

- Navegador moderno con WebGL (Chrome, Firefox)
- Python 3 para lanzar el servidor local (opcional) o servidor estático

Ejecución rápida

1. Abrir terminal en la carpeta del proyecto.
2. Ejecutar:

```
python server-launcher.py
```

3. Abrir `http://localhost:8000` en el navegador.
4. Entrar en la carpeta del juego `Laberinto/`.

Controles y atajos

- Acción / Tecla

Acción	Tecla / Control	Descripción
Avanzar	W	Movimiento hacia delante
Retroceder	S	Movimiento hacia atrás
Girar izquierda	A	Rotación del jugador
Girar derecha	D	Rotación del jugador
Mirar	Ratón	Control de la cámara en primera persona
Recoger pick-up	Click izquierdo	Recoge el objeto si está suficientemente cerca
Abrir puerta	Click izquierdo sobre pomo	Abre si se poseen todos los pick-ups
Vista superior (minimapa)	M	Activa/desactiva el minimapa
Zoom minimapa	Rueda del ratón	Acerca o aleja la cámara superior
Reiniciar partida	- (recargar página)	Recarga la partida

11. Resumen de requisitos cumplidos

Requisito	Implementación
4 Pick-ups	Farolillo, Abanico, Castañuelas, Rebujito
Llave	Farolillo actúa como llave
Objeto articulado	Abanico (modelo jerárquico)
Revolución	Farolillo, Castañuelas, Rebujito
Extrusión	Farolillo, Abanico, Castañuelas
Barrido	Abanico (varillas), pajita del rebujito
CSG	Farolillo, Castañuelas, Rebujito (huecos)
Animación continua	Abanico
Puerta animada	Sí (llave + hoja)
Cámara primera persona	Sí
Cámara superior	Sí
Materiales con color	Sí
Materiales con textura	Sí
Mapa de relieve (normal map)	Parcial (textura generada por canvas)
Luces dinámicas	Sí
Recogida de pick-ups	Sí
Apertura condicionada de puerta	Sí

12. Carpeta final recomendada

```

Entrega_Practica_SG/
├── codigo/
│   ├── juego/ (código principal: Laberinto/ MyScene.js ...)
│   ... (file continues)

```